

PROMPT MASTER — TRAMPAÍ MVP-1

Instruções para o agente IA (Antigravity/Gemini): Este documento é uma especificação executável. Implemente exatamente como descrito. NÃO invente features além das listadas. NÃO use bibliotecas além das declaradas. Quando houver ambiguidade, pergunte ao usuário antes de codar. Execute as tasks na ordem listada na seção 9. Cada task tem critério de aceitação — só avance quando o critério estiver atendido.

1. CONTEXTO DO PRODUTO

Nome: Trampaí **Slogan:** “Faz tua grana, no teu tempo, do teu jeito.” **Tipo:** SaaS/Marketplace web (PWA + TWA Play Store) de serviços locais **Modelo de negócio:** Pay-per-Lead via PIX. Cliente posta serviço grátis. Prestador paga créditos pra ver o WhatsApp do cliente. **Região:** Guapó-GO + raio de 150km (filtro geográfico via Google Maps Geocoding) **Público:** Pessoa Física no Brasil (CPF, telefone BR, PIX) **Desenvolvedor:** Solo, sem time. Código deve ser autoexplicativo, com comentários em PT-BR nas regras de negócio.

2. STACK TÉCNICA — DECISÕES FIXAS (não negociar)

Camada	Tecnologia	Versão alvo
Framework	Next.js (App Router)	15.x
Linguagem	TypeScript estrito	5.x
Estilo	Tailwind CSS + Shadcn/ui	latest
ORM	Prisma	5.x
Banco	PostgreSQL via Supabase	-
Storage	Supabase Storage	-
Auth	NextAuth.js v5 (Auth.js) — Google OAuth único	5.x
Validação	Zod	3.x

Gateway PIX	InfinityPay (PF/CPF)	API REST
Email	Resend	latest
Push	Web Push API + Service Worker (web-push lib)	-
Mapas	Google Maps JS API + Geocoding API + Places API	-
Rate Limit	Upstash Redis	-
Forms	React Hook Form + Zod resolver	-
Máscaras	react-imask	-
Ícones	lucide-react	-
Deploy	Docker + docker-compose na VPS IntegraHost	-
CI/CD	GitHub Actions (build + push image + ssh deploy)	-

Proibido:

- Bibliotecas além das listadas acima (a menos que peça permissão)
- `any` no TypeScript (use `unknown` + `narrow`)
- `useEffect` para data fetching (use Server Components / Server Actions)
- API Routes quando Server Action resolve
- `Float` para dinheiro (use `Int` em centavos)
- `localStorage` para dados sensíveis
- Mocks ou seeders de produção (apenas script `pnpm seed:dev` opcional, fora do build prod)

3. IDENTIDADE VISUAL

Paleta (Tailwind config)

```
// tailwind.config.ts - extend.colors
brand: {
  primary: "#21284E", // Azul marinho - texto, headers, base
  accent: "#F69926", // Laranja - CTAs, botões, destaques
```

```
cyan: "#5EB4B8",          // Ciano – gradiente, ícones secundários
cream: "#F7EFCF",         // Creme – backgrounds suaves, slogan
}
```

Background padrão das telas públicas

```
bg-gradient-to-br from-brand-cyan to-brand-cream
```

Tipografia

- Fonte: **Inter** (next/font/google)
- Headings: `font-bold text-brand-primary`
- CTA buttons: `bg-brand-accent text-white hover:bg-brand-accent/90`

Splash Screen — Especificação Pixel-Perfect

- Fundo: `bg-brand-primary (#21284E)` full screen
- Título “Trapaí”: `text-white text-5xl font-bold`
- Slogan “Faz tua grana, no teu tempo, do teu jeito.”: `text-brand-cream animate-pulse text-lg mt-4`
- Duas engrenagens `lucide-react` (`Cog` ou `Settings`):
 - Engrenagem 1: cor `#F69926` , gira sentido horário (`animate-spin`), tamanho 64px
 - Engrenagem 2: cor `#5EB4B8` , gira sentido anti-horário (custom keyframe `animate-spin-reverse`), tamanho 64px
 - Posicionadas lado a lado abaixo do slogan
- Duração: 2.5s, fade-out suave
- Aparece **toda vez que o app abre** (não só primeiro acesso)
- Componente: `<SplashScreen />` em `components/splash-screen.tsx`
- Controle via Context: `SplashProvider` envolve o `RootLayout`

Animação custom no `globals.css`

```
@keyframes spin-reverse {
  from { transform: rotate(0deg); }
  to { transform: rotate(-360deg); }
```

```

}
.animate-spin-reverse {
  animation: spin-reverse 2s linear infinite;
}

```

4. ESTRUTURA DE PASTAS (criar exatamente assim)

```

trampai/
├── .github/workflows/
│   └── deploy.yml
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── public/
│   ├── manifest.json
│   ├── sw.js
│   ├── icons/ (192, 512, maskable)
│   └── .well-known/assetlinks.json (TWA)
├── src/
│   ├── app/
│   │   ├── (public)/
│   │   │   ├── page.tsx # Landing
│   │   │   ├── login/page.tsx
│   │   │   └── termos/page.tsx
│   │   ├── (cliente)/
│   │   │   ├── dashboard/page.tsx
│   │   │   ├── novo-servico/page.tsx
│   │   │   └── meus-servicos/page.tsx
│   │   ├── (prestador)/
│   │   │   ├── mural/page.tsx
│   │   │   ├── meus-leads/page.tsx
│   │   │   ├── carteira/page.tsx # Recarga PIX
│   │   │   ├── perfil/page.tsx
│   │   │   └── verificacao/page.tsx # Upload doc
│   │   ├── (admin)/
│   │   │   ├── admin/page.tsx # Dashboard
│   │   │   ├── admin/usuarios/page.tsx
│   │   │   ├── admin/transacoes/page.tsx
│   │   │   ├── admin/verificacoes/page.tsx
│   │   │   └── admin/auditoria/page.tsx
│   │   └── api/
│   │       ├── auth/[...nextauth]/route.ts
│   │       ├── webhooks/infinitypay/route.ts
│   │       └── push/subscribe/route.ts

```

```

|   |   |   └─ health/route.ts
|   |   └─ layout.tsx
|   |   └─ globals.css
|   |   └─ manifest.ts
|   └─ components/
|       |   └─ ui/                                # Shadcn
|       |   └─ splash-screen.tsx
|       |   └─ service-card.tsx
|       |   └─ credit-package-card.tsx
|       |   └─ custom-credit-input.tsx
|       |   └─ service-form.tsx
|       |   └─ unlock-button.tsx
|       |   └─ review-form.tsx
|       |   └─ report-button.tsx
|       |   └─ nav/
|       |       |   └─ client-nav.tsx
|       |       |   └─ provider-nav.tsx
|       |       └─ admin-nav.tsx
|   └─ server/
|       |   └─ actions/
|       |       |   └─ service.ts                # createService, closeService
|       |       |   └─ unlock.ts                # unlockService
|       |       |   └─ credits.ts               # buyCredits, addManualCredits
|       |       |   └─ review.ts
|       |       |   └─ report.ts
|       |       └─ verification.ts              # submit + approve
|       |       └─ admin.ts                     # banUser, dashboardStats
|       |   └─ auth.ts                          # NextAuth config
|       |   └─ db.ts                            # Prisma singleton
|       |   └─ infinitypay.ts                   # SDK wrapper
|       |   └─ resend.ts
|       |   └─ push.ts
|       |   └─ supabase.ts
|       |   └─ geocode.ts                       # Google Maps wrapper
|       |   └─ rate-limit.ts                   # Upstash
|       |   └─ audit.ts                         # log helper
|   └─ lib/
|       |   └─ constants.ts                     # PACKAGES, MIN_CUSTOM_CREDITS, etc.
|       |   └─ masks.ts                       # phone, cep, cpf
|       |   └─ viacep.ts
|       |   └─ distance.ts                     # haversine
|       |   └─ utils.ts                        # cn() do shadcn
|       |   └─ validations/
|       |       |   └─ service.ts
|       |       |   └─ credit.ts
|       |       └─ user.ts
|   └─ types/

```

```

|   |   └─ next-auth.d.ts
|   |   └─ index.ts
|   └─ middleware.ts # rota guards + rate limit
└─ .env.example
└─ .gitignore
└─ .dockerignore
└─ Dockerfile
└─ docker-compose.yml
└─ next.config.ts
└─ tailwind.config.ts
└─ tsconfig.json
└─ package.json
└─ README.md

```

5. SCHEMA PRISMA COMPLETO

Arquivo `prisma/schema.prisma` — copie EXATAMENTE:

```

// -----
// Trampai - Schema Prisma
// Banco inicia 100% vazio. Sem seeders de produção.
// Dinheiro em centavos (Int). Créditos em unidades inteiras (Int).
// -----

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")
}

// ===== ENUMS =====

enum Role {
  CLIENT
  PROVIDER
  ADMIN
}

enum ServiceStatus {
  OPEN // Aceitando interessados

```

```

    LOCKED      // Desbloqueio exclusivo ativo (oculto do mural por 24h)
    CLOSED      // Cliente marcou como resolvido
    EXPIRED      // 7 dias sem fechar
    REMOVED      // Removido por admin (denúncia precedente)
}

```

```

enum UnlockType {
    NORMAL      // 1 crédito - serviço continua no mural
    EXCLUSIVE    // 3 créditos - esconde do mural por 24h
}

```

```

enum TransactionType {
    PURCHASE      // Compra de créditos via PIX
    UNLOCK_SPEND   // Gasto em desbloqueio
    REFUND_ANTIVACUO // Estorno automático anti-vácuo
    WELCOME_BONUS  // 5 créditos grátis no cadastro (expira 7 dias)
    ADMIN_GRANT     // Crédito manual admin
    ADMIN_REVOKE    // Débito manual admin
    REFERRAL_BONUS  // Indicação confirmada
}

```

```

enum VerificationStatus {
    PENDING
    APPROVED
    REJECTED
}

```

```

enum ReportStatus {
    OPEN
    REVIEWED_VALID
    REVIEWED_INVALID
}

```

```

enum NotificationType {
    NEW_SERVICE_NEARBY
    LEAD_UNLOCKED
    SERVICE_CLOSED
    REVIEW_RECEIVED
    CREDITS_ADDED
    VERIFICATION_RESULT
    WELCOME_BONUS_EXPIRING
}

```

```

// ===== NEXTAUTH =====

```

```

model Account {
    id              String @id @default(cuid())
}

```

```

    userId          String
    type            String
    provider        String
    providerAccountId String
    refresh_token   String? @db.Text
    access_token    String? @db.Text
    expires_at      Int?
    token_type      String?
    scope           String?
    id_token        String? @db.Text
    session_state   String?
    user            User      @relation(fields: [userId], references: [id], onDelete

    @@unique([provider, providerAccountId])
}

model Session {
  id          String    @id @default(cuid())
  sessionToken String    @unique
  userId      String
  expires     DateTime
  user        User      @relation(fields: [userId], references: [id], onDelete: C
}

model VerificationToken {
  identifier String
  token      String    @unique
  expires    DateTime

  @@unique([identifier, token])
}

// ===== USUÁRIO =====

model User {
  id          String    @id @default(cuid())
  name        String?
  email       String    @unique
  emailVerified DateTime?
  image       String?

  // Identidade Brasil
  phone      String?    // formato +5564991234567 (E.164)
  cpf        String?    @unique
  birthDate  DateTime?

  // Role e status

```

```

role          Role          @default(CLIENT)
isBanned       Boolean      @default(false)
banReason      String?
acceptedTerms Boolean      @default(false)
acceptedTermsAt DateTime?

// Carteira (créditos em unidades inteiras)
creditBalance Int           @default(0)

// Bônus de boas-vindas (5 créditos com 7 dias)
welcomeBonusGrantedAt DateTime?
welcomeBonusExpiresAt DateTime?
welcomeBonusUsedCredits Int @default(0)

// Geolocalização padrão
cep            String?
street         String?
number        String?
neighborhood   String?
city           String?
state          String?
latitude       Float?
longitude      Float?

// Push notifications
pushSubscriptions PushSubscription[]

// Verificação de prestador
providerVerification ProviderVerification?

// Perfil prestador
providerProfile      ProviderProfile?

// Indicação
referralCode String? @unique
referredById String?
referredBy      User? @relation("Referrals", fields: [referredById], referenc
referrals       User[] @relation("Referrals")

// Relations
accounts        Account[]
sessions        Session[]
servicesAsClient Service[] @relation("ClientServices")
unlocks         LeadUnlock[] @relation("ProviderUnlocks")
transactions    Transaction[]
reviewsGiven    Review[] @relation("ReviewAuthor")
reviewsReceived Review[] @relation("ReviewTarget")

```

```

reportsMade      Report[]      @relation("Reporter")
reportsAgainst   Report[]      @relation("Reported")
auditLogs        AuditLog[]
notifications    Notification[]

createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
deletedAt DateTime?

@@index([role])
@@index([city])
@@index([latitude, longitude])
}

model ProviderProfile {
  id          String    @id @default(cuid())
  userId      String    @unique
  user        User      @relation(fields: [userId], references: [id], onDelete: C
  bio         String?   @db.Text
  yearsExp    Int?
  categories  String[]  // sem limite - usuário decide
  serviceAreas String[] // CEPs ou bairros que atende; vazio = atende tudo
  serviceRadiusKm Int    @default(150)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model ProviderVerification {
  id          String    @id @default(cuid())
  userId      String    @unique
  user        User      @relation(fields: [userId], references: [id], on
  documentUrl String    // URL no Supabase Storage (RG/CNH)
  selfieUrl   String    // URL da selfie
  status      VerificationStatus @default(PENDING)
  reviewedById String?
  reviewedAt  DateTime?
  rejectReason String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([status])
}

// ===== SERVIÇO =====

model Service {
  id          String    @id @default(cuid())

```

```

    clientId      String
    client         User          @relation("ClientServices", fields: [clientId], ref

    title         String
    description    String        @db.Text
    category       String        // texto livre OU da lista fixa (ver constants.ts)
    imageUrls      String[]      // máx 3, Supabase Storage

    // Localização
    cep            String
    street         String?
    number         String?
    neighborhood   String
    city           String
    state          String        @default("GO")
    latitude       Float?
    longitude      Float?

    // Status & lifecycle
    status         ServiceStatus @default(OPEN)
    lockedUntil    DateTime?     // quando volta a aparecer no mural após EXCLUSIVE
    expiresAt      DateTime      // createdAt + 7 dias
    closedAt       DateTime?

    unlocks        LeadUnlock[]
    reports        Report[]
    reviews        Review[]

    createdAt      DateTime @default(now())
    updatedAt      DateTime @updatedAt

    @@index([status, city])
    @@index([category])
    @@index([latitude, longitude])
    @@index([expiresAt])
    @@index([lockedUntil])
}

// ===== DESBLOQUEIO =====

model LeadUnlock {
    id            String        @id @default(cuid())
    serviceId     String
    service       Service       @relation(fields: [serviceId], references: [id], onDelete
    providerId    String
    provider      User          @relation("ProviderUnlocks", fields: [providerId], refer

```

```

type            UnlockType
creditsSpent Int           // 1 ou 3

// Estorno anti-vácuo
refunded        Boolean    @default(false)
refundedAt      DateTime?
refundReason    String?

// Vinculação à transação de débito (auditoria)
transactionId   String?    @unique
transaction     Transaction? @relation(fields: [transactionId], references: [id])

createdAt       DateTime @default(now())

@@unique([serviceId, providerId]) // prestador só desbloqueia 1x o mesmo serviço
@@index([providerId])
@@index([serviceId])
}

// ===== TRANSAÇÕES =====

model Transaction {
    id            String          @id @default(cuid())
    userId        String
    user          User            @relation(fields: [userId], references: [id], onD

    type          TransactionType
    credits        Int            // positivo = entrada, negativo = saída
    amountCents    Int            @default(0) // valor em R$ centavos (só PURCHASE)

    // PIX (apenas PURCHASE)
    pixTxId        String?        @unique // ID da transação no InfinityPay (idempo
    pixQrCode      String?        @db.Text
    pixCopyPaste   String?        @db.Text
    pixExpiresAt   DateTime?
    paidAt         DateTime?

    description    String?

    unlock          LeadUnlock?

    createdAt       DateTime @default(now())
    updatedAt       DateTime @updatedAt

    @@index([userId, type])
    @@index([pixTxId])
    @@index([createdAt])

```

```

}

// ===== REVIEWS =====

model Review {
    id          String    @id @default(cuid())
    serviceId   String
    service     Service   @relation(fields: [serviceId], references: [id], onDelete:

    authorId    String
    author      User      @relation("ReviewAuthor", fields: [authorId], references:
    targetId    String
    target      User      @relation("ReviewTarget", fields: [targetId], references:

    rating      Int       // 1-5
    comment     String?   @db.Text
    isPublic    Boolean   @default(false) // só vira true quando os 2 avaliarem OU a

    createdAt  DateTime @default(now())

    @@unique([serviceId, authorId, targetId])
    @@index([targetId, isPublic])
}

// ===== DENÚNCIA =====

model Report {
    id          String    @id @default(cuid())
    reporterId  String
    reporter    User      @relation("Reporter", fields: [reporterId], references

    // Pode reportar usuário OU serviço (um dos dois)
    reportedUserId String?
    reportedUser  User?    @relation("Reported", fields: [reportedUserId], refere
    serviceId     String?
    service       Service? @relation(fields: [serviceId], references: [id])

    reason       String
    description  String?   @db.Text
    status       ReportStatus @default(OPEN)
    reviewedAt   DateTime?
    reviewNotes  String?

    createdAt  DateTime @default(now())

    @@index([status])
}

```

```
// ===== NOTIFICAÇÕES =====
```

```
model PushSubscription {
  id          String @id @default(cuid())
  userId      String
  user        User    @relation(fields: [userId], references: [id], onDelete: Cascad
  endpoint    String @unique
  p256dh      String
  auth        String
  userAgent   String?
  createdAt   DateTime @default(now())
}
```

```
model Notification {
  id          String          @id @default(cuid())
  userId      String
  user        User            @relation(fields: [userId], references: [id], onDelete:
  type        NotificationType
  title       String
  body        String
  data        Json?
  read        Boolean          @default(false)
  sentPush    Boolean          @default(false)
  sentEmail   Boolean          @default(false)
  createdAt   DateTime          @default(now())

  @@index([userId, read])
}
```

```
// ===== AUDITORIA =====
```

```
model AuditLog {
  id          String @id @default(cuid())
  actorId     String?
  actor       User?   @relation(fields: [actorId], references: [id])
  action      String  // ex: "ADMIN_GRANT_CREDITS", "USER_BANNED"
  entity      String  // ex: "User", "Service"
  entityId    String?
  metadata    Json?
  ip          String?
  createdAt   DateTime @default(now())

  @@index([actorId])
  @@index([entity, entityId])
  @@index([createdAt])
}
```

6. CONSTANTES E REGRAS DE NEGÓCIO (lib/constants.ts)

```
// src/lib/constants.ts

export const CREDIT_PACKAGES = [
  { id: "basico",      label: "Básico",      credits: 30, priceCents: 2999 },
  { id: "profissional", label: "Profissional", credits: 50, priceCents: 4999 },
  { id: "prime",       label: "Prime",       credits: 100, priceCents: 9999 },
] as const;

export const CUSTOM_CREDIT_RULES = {
  MIN_CREDITS: 10,
  PRICE_CENTS_PER_CREDIT: 100, // R$ 1,00 fixo
} as const;

export const UNLOCK_COSTS = {
  NORMAL: 1,
  EXCLUSIVE: 3,
} as const;

export const EXCLUSIVE_LOCK_HOURS = 24;
export const ANTIVACUO_WINDOW_HOURS = 2;
export const SERVICE_EXPIRATION_DAYS = 7;
export const WELCOME_BONUS_CREDITS = 5;
export const WELCOME_BONUS_EXPIRATION_DAYS = 7;
export const REVIEW_BLIND_DAYS = 3;
export const MAX_SERVICE_IMAGES = 3;
export const SERVICE_RADIUS_KM_DEFAULT = 150;

// Categorias sugeridas (cliente também pode digitar livre)
export const SUGGESTED_CATEGORIES = [
  "Pedreiro",
  "Eletricista",
  "Encanador",
  "Diarista",
  "Pintor",
  "Jardineiro",
  "Marceneiro",
  "Chaveiro",
  "Mecânico",
  "Mudanças/Frete",
  "Refrigeração/Ar-condicionado",
  "Técnico de Informática",
```

```

    "Costureira",
    "Cuidador",
    "Outros",
] as const;

export const PIX_EXPIRATION_MINUTES = 30;

```

Fórmula matemática — Compra avulsa

Validação: $\text{quantity} \geq 10$ && quantity é Inteiro
 $\text{priceCents} = \text{quantity} * 100$
 Exemplo: 12 créditos $\rightarrow 12 * 100 = 1200$ centavos = R\$ 12,00

Fluxo Anti-Vácuo (matemática)

Trigger: `closeService(serviceId)`

Para cada `LeadUnlock` onde:

- `serviceId` = atual
- `type` = NORMAL
- `refunded` = false
- `createdAt` > (NOW() - 2 horas)

Executar dentro de `prisma.$transaction`:

1. Criar Transaction(`type=REFUND_ANTIVACUO`, `credits+=1`, `userId=providerId`)
2. `user.creditBalance += 1`
3. `unlock.refunded = true`, `refundedAt = NOW()`, `refundReason = "anti-vacuo"`
4. Notificar provider (push + email)
5. AuditLog

Fluxo Desbloqueio (matemática)

`unlockService(serviceId, providerId, type)`:
 $\text{cost} = \text{type} === \text{NORMAL} ? 1 : 3$

Validar:

- `service.status === OPEN`
- `service.expiresAt > NOW()`
- `provider.creditBalance >= cost`
- `!provider.isBanned`
- Não existe `LeadUnlock` prévio (provider+service)
- Provider verificado (`verification.status === APPROVED`)

```
Em prisma.$transaction (com SELECT FOR UPDATE no user):
  1. Criar Transaction(type=UNLOCK_SPEND, credits=-cost)
  2. user.creditBalance -= cost
  3. Criar LeadUnlock(transactionId=txId)
  4. Se EXCLUSIVE:
    - service.status = LOCKED
    - service.lockedUntil = NOW() + 24h
  5. Notificar cliente (push + email): "Novo prestador interessado"
  6. AuditLog

Retornar: { whatsappLink: "https://wa.me/55<DDD><numero>?text=..." }
```

Job/cron alternativa (sem cron):

Em toda query do mural, filtrar:

```
WHERE status = OPEN
```

```
OR (status = LOCKED AND lockedUntil < NOW())
```

E ao retornar LOCKED expirado, atualizar pra OPEN no mesmo request.

Bônus de Boas-Vindas

No callback de signIn() do NextAuth, primeira vez (welcomeBonusGrantedAt = null):

- creditBalance += 5
- welcomeBonusGrantedAt = NOW()
- welcomeBonusExpiresAt = NOW() + 7 dias
- Criar Transaction(type=WELCOME_BONUS, credits=+5)
- Gerar referralCode único (6 chars alfanuméricos)

Toda vez que UNLOCK_SPEND ocorre:

Se welcomeBonusExpiresAt > NOW() && welcomeBonusUsedCredits < 5:

```
welcomeBonusUsedCredits += custo
```

Quando welcomeBonusExpiresAt < NOW():

Os créditos restantes não-usados expiram:

```
creditsExpired = 5 - welcomeBonusUsedCredits
```

Se creditsExpired > 0:

```
creditBalance -= creditsExpired (mas não negativo)
```

```
Transaction(type=ADMIN_REVOKE, credits=-creditsExpired, description="Bônus
```

→ Implementar via query no middleware: ao carregar user, checar e expirar.

→ SEM cron job. Tudo lazy na próxima requisição do user.

Programa de Indicação

Cada user tem referralCode único.

Novo user pode informar referralCode no onboarding.

Quando o indicado faz a PRIMEIRA compra de créditos (Transaction PURCHASE confirm

- referredBy.creditBalance += 10
- Transaction(type=REFERRAL_BONUS, credits=+10, userId=referredById)
- Notificar quem indicou

7. AUTENTICAÇÃO — NextAuth v5 + Google OAuth ÚNICO

```
// src/server/auth.ts (esqueleto - implementar)

import NextAuth from "next-auth";
import Google from "next-auth/providers/google";
import { PrismaAdapter } from "@auth/prisma-adapter";
import { db } from "../db";

export const { handlers, auth, signIn, signOut } = NextAuth({
  adapter: PrismaAdapter(db),
  providers: [Google({
    profile(profile) {
      return {
        id: profile.sub,
        name: profile.name,
        email: profile.email,
        image: profile.picture,
        emailVerified: profile.email_verified ? new Date() : null,
      };
    },
  })],
  session: { strategy: "database" },
  callbacks: {
    async session({ session, user }) {
      session.user.id = user.id;
      session.user.role = user.role;
      session.user.creditBalance = user.creditBalance;
      session.user.isBanned = user.isBanned;
      return session;
    },
    async signIn({ user, account, profile }) {
      // Bloquear banidos
      if (user.id) {
        const dbUser = await db.user.findUnique({ where: { id: user.id } });
        if (dbUser?.isBanned) return false;
      }
      return true;
    },
  },
});
```

```

    },
  },
  events: {
    async createUser({ user }) {
      // Conceder bônus de boas-vindas + gerar referralCode
      // ... implementar
    },
  },
});

// Estender tipos:
// src/types/next-auth.d.ts
declare module "next-auth" {
  interface Session {
    user: {
      id: string;
      role: "CLIENT" | "PROVIDER" | "ADMIN";
      creditBalance: number;
      isBanned: boolean;
    } & DefaultSession["user"];
  }
}

```

Onboarding pós-login

Após primeiro login, redirecionar para `/onboarding` onde o user:

1. Escolhe `role`: CLIENT ou PROVIDER (ADMIN é setado manualmente no banco)
2. Confirma idade ≥ 18 (checkbox obrigatório)
3. Aceita termos de uso (checkbox obrigatório)
4. Preenche telefone (máscara `(XX) 9XXXX-XXXX`)
5. Preenche CEP (autopreenche endereço via ViaCEP)
6. Opcional: insere `referralCode` de quem o indicou
7. Se PROVIDER: vai para tela de upload de documento + selfie (status PENDING até admin aprovar)

Sem completar onboarding, não consegue acessar nenhuma rota autenticada (middleware bloqueia).

8. INTEGRAÇÃO INFINITYPAY — PIX

Atenção: InfinityPay tem documentação pública limitada. Como o desenvolvedor já tem conta InfinityPay com CPF, **assume-se que ele forneça as credenciais e a base URL no .env**. Se a API REST do InfinityPay não suportar webhook PIX programático, **implementar polling de fallback** (consultar status a cada 30s por 30min após criação).

```
// src/server/infinitypay.ts (wrapper)

interface CreatePixChargeInput {
  userId: string;
  amountCents: number;
  description: string;
  expiresInMinutes: number;
}

interface PixCharge {
  txId: string;          // ID único do gateway
  qrCode: string;        // imagem base64
  copyPaste: string;     // código PIX copia-e-cola
  expiresAt: Date;
}

export async function createPixCharge(input: CreatePixChargeInput): Promise<PixCh
  // POST https://api.infinitypay.io/...
  // Idempotency-Key: input.userId + timestamp
  // Retornar txId, qrCode, copyPaste
}

export async function getPixStatus(txId: string): Promise<"PENDING" | "PAID" | "E
  // GET status
}

export function verifyWebhookSignature(payload: string, signature: string): boole
  // HMAC SHA256 com WEBHOOK_SECRET
}
```

Webhook handler — IDEMPOTENTE

```
// src/app/api/webhooks/infinitypay/route.ts

export async function POST(req: Request) {
  // 1. Validar assinatura (HMAC)
  const signature = req.headers.get("x-infinitypay-signature");
```

```

const rawBody = await req.text();
if (!verifyWebhookSignature(rawBody, signature)) {
  return new Response("Invalid signature", { status: 401 });
}

const event = JSON.parse(rawBody);

if (event.type !== "pix.paid") {
  return new Response("OK (ignored)", { status: 200 });
}

const txId = event.data.txId;

// 2. IDEMPOTÊNCIA – buscar Transaction com pixTxId
const tx = await db.transaction.findUnique({ where: { pixTxId: txId } });
if (!tx) return new Response("Transaction not found", { status: 404 });
if (tx.paidAt) return new Response("Already processed", { status: 200 }); // ✅

// 3. Em prisma.$transaction:
//   - tx.paidAt = NOW()
//   - user.creditBalance += tx.credits
//   - Notificar user (push + email)
//   - Se for primeira compra do user E ele tiver referredById:
//       conceder REFERRAL_BONUS para o indicador
//   - AuditLog

return new Response("OK", { status: 200 });
}

```

9. TASKS DE EXECUÇÃO — ORDEM OBRIGATÓRIA

Execute uma de cada vez. **Não avance sem o critério de aceitação.** Se uma task tiver dúvida, pare e pergunte.

TASK 01 — Setup do projeto

- `pnpm create next-app trampai --typescript --tailwind --app --src-dir --import-alias "@/*"`
- **Instalar dependências:** `prisma @prisma/client next-auth@beta @auth/prisma-adapter zod react-hook-form @hookform/resolvers resend web-push @upstash/ratelimit @upstash/redis @supabase/supabase-js react-imask lucide-react`
- **Instalar shadcn:** `pnpm dlx shadcn@latest init (cores: brand-primary)`

- Adicionar componentes shadcn: `button card input label dialog dropdown-menu form select textarea toast badge avatar tabs`
- Criar estrutura de pastas conforme seção 4
- Configurar `tailwind.config.ts` com paleta seção 3
- Configurar `next.config.ts` com `output: "standalone"` (Docker)
- Criar `.env.example` com todas variáveis (ver seção 11)
- **Aceitação:** `pnpm dev` roda sem erro, `/` mostra página em branco com gradiente.

TASK 02 — Prisma + Supabase

- Copiar `schema.prisma` da seção 5 EXATAMENTE
- Configurar Supabase project (usuário criou conta) e pegar `DATABASE_URL` + `DIRECT_URL`
- `pnpm prisma migrate dev --name init`
- Criar `src/server/db.ts` com singleton Prisma
- **Aceitação:** `pnpm prisma studio` abre e mostra todas as tabelas vazias.

TASK 03 — Auth (NextAuth v5 + Google)

- Configurar `src/server/auth.ts` (seção 7)
- Criar `src/types/next-auth.d.ts`
- Criar `src/app/api/auth/[...nextauth]/route.ts`
- Criar `src/middleware.ts` com guards:
 - `/cliente/*` → role CLIENT
 - `/prestador/*` → role PROVIDER (e verification APPROVED para mural)
 - `/admin/*` → role ADMIN
 - Redirecionar para `/onboarding` se faltar dados
- Criar página `/login` com botão “Entrar com Google”
- Implementar event `createUser` que concede WELCOME_BONUS e gera `referralCode`
- **Aceitação:** login com Google funciona, user é criado no banco com 5 créditos e Transaction WELCOME_BONUS.

TASK 04 — Onboarding

- Criar `/onboarding` com formulário multi-step:
 - Step 1: escolher role (CLIENT/PROVIDER)
 - Step 2: dados pessoais (nome, telefone com máscara, CEP+endereço via ViaCEP)
 - Step 3: idade ≥ 18 + termos (links pra `/termos`)
 - Step 4: opcional referralCode
 - Step 5 (só PROVIDER): upload de documento + selfie via Supabase Storage → cria `ProviderVerification(PENDING)`
- Server Action `completeOnboarding(data)` com validação Zod
- Criar `/termos` com placeholder do texto (TODO: copy real)
- **Aceitação:** após login, user é forçado a completar onboarding antes de acessar dashboard.

TASK 05 — Splash Screen + Layout base + PWA manifest

- Criar `<SplashScreen />` conforme seção 3
- Criar `SplashProvider` com state de visibilidade (toda vez que app abre)
- Criar `app/manifest.ts` (PWA)
- Criar ícones PWA em `public/icons/`
- Configurar `public/sw.js` (service worker básico — push handler virá depois)
- Layout raiz com `<body className="bg-gradient-to-br from-brand-cyan to-brand-cream">`
- **Aceitação:** abrir o app mostra splash 2.5s com engrenagens girando em sentidos opostos. Lighthouse PWA passa.

TASK 06 — Cliente: Postar Serviço

- Página `/cliente/novo-servico` com `<ServiceForm />`
- Campos: título, categoria (combobox: lista fixa + opção "Outros: digite"), descrição, CEP→endereço, bairro, até 3 imagens (Supabase Storage)
- Server Action `createService(data)` :

- Validação Zod
- Geocoding via Google Maps API (lat/lng do endereço)
- Verificar limite: cliente não pode ter mais de **3 serviços OPEN simultaneamente**
- Verificar cidade dentro do raio 150km de Guaporé-GO (lat -16.8270, lng -49.5340) via Haversine
- Upload de imagens (máx 3, jpeg/png, ≤2MB cada)
- Criar Service com `expiresAt = NOW() + 7 dias`
- Disparar push notifications para prestadores na categoria + raio (job inline)
- Página `/cliente/meus-servicos` lista os serviços com botão "Marcar como Resolvido" (chama `closeService`)
- **Aceitação:** cliente posta serviço, aparece em "meus-servicos", outros prestadores na região recebem push.

TASK 07 — Prestador: Mural + Desbloqueio

- Página `/prestador/mural` com filtros:
 - Categoria (multi)
 - Raio em km (slider 5-150)
 - Ordenar por: mais recente / mais perto
- Query Prisma:


```
WHERE (status = OPEN OR (status = LOCKED AND lockedUntil < NOW()))
      AND expiresAt > NOW()
      AND categoria IN (filtros)
      AND distancia(provider.lat/lng, service.lat/lng) <= radius
```
- Renderizar `<ServiceCard />`:
 - Mostra: título, categoria, bairro+cidade, descrição truncada, miniatura da 1ª imagem se houver, "há X horas"
 - **NÃO mostra:** nome do cliente, telefone, endereço completo
 - Botões: "Desbloquear (1 crédito)" e "Exclusivo (3 créditos)"
- Server Action `unlockService(serviceId, type)` conforme seção 6

- Após sucesso: modal com botão "Abrir WhatsApp" (`https://wa.me/55XXXXXXXXXXXX?text=Olá! Vim pelo Trampai sobre seu serviço: <título>`)
- **Aceitação:** prestador desbloqueia, crédito debita, WhatsApp link funciona, exclusive esconde do mural por 24h.

TASK 08 — Carteira: Compra de créditos via PIX

- Página `/prestador/carteira` com:
 - Saldo atual em destaque
 - 3 cards de pacotes fixos (`<CreditPackageCard />`)
 - Card de compra avulsa (`<CustomCreditInput />`) — input numérico com validação min 10, mostra "R\$ X,XX" em tempo real
- Server Action `buyCredits(quantity)` :
 - Se `quantity` corresponde a um pacote: usar `priceCents` do pacote
 - Senão: validar `quantity >= 10`, calcular `priceCents = quantity * 100`
 - Chamar `createPixCharge()` do InfinityPay
 - Criar Transaction(PURCHASE, `pixTxId`, `pixQrCode`, `pixCopyPaste`, `pixExpiresAt`) com `paidAt = null`
 - Retornar `{ qrCode, copyPaste, expiresAt }`
- UI mostra QR Code + copia-e-cola + countdown 30min + polling do status a cada 5s
- **Aceitação:** prestador gera PIX, vê QR Code. (pagamento real testado no próximo task)

TASK 09 — Webhook InfinityPay (idempotente)

- `src/app/api/webhooks/infinitypay/route.ts` conforme seção 8
- Validação HMAC com `INFINITYPAY_WEBHOOK_SECRET`
- Idempotência via `pixTxId`
- Após confirmação: créditos somados, push+email enviados, REFERRAL_BONUS aplicado se for primeira compra
- **Aceitação:** simular webhook com curl funciona. Reenviar mesmo evento não duplica créditos.

TASK 10 — Anti-Vácuo

- Server Action `closeService(serviceId)` conforme seção 6
- Validação: `service.client.id === user.id`
- Lógica matemática exata
- Notificar prestadores estornados (push + email)
- **Aceitação:** cliente fecha serviço em <2h, prestador NORMAL recebe estorno automaticamente, balance volta +1.

TASK 11 — Reviews bidirecionais

- Após `closeService` : criar pendência de review pra cliente E prestador(es) que desbloquearam
- Páginas com formulário (estrelas 1-5 + comentário opcional)
- Server Action `submitReview(serviceId, targetId, rating, comment)`
- Lógica de "isPublic":
 - Se ambos avaliaram: `isPublic = true` para ambos
 - Job lazy: ao consultar, se `createdAt + 3 dias < NOW()` E ainda não público, marcar como público
- Mostrar reviews públicos no perfil do prestador
- **Aceitação:** cliente fecha, ambos podem avaliar, avaliação aparece após ambos OU 3 dias.

TASK 12 — Sistema de denúncia

- Botão `<ReportButton />` em cards de serviço e perfil de user
- Modal com motivo (select) + descrição
- Server Action `submitReport(target, reason, description)`
- Rate limit: 5 denúncias por user por dia (Upstash)
- **Aceitação:** denúncia é criada com status OPEN, aparece no painel admin.

TASK 13 — Push notifications

- Tela de "ativar notificações" no primeiro acesso ao dashboard (cliente e prestador)

- Endpoint `/api/push/subscribe` salva `PushSubscription`
- Helper `src/server/push.ts` com `sendPush(userId, title, body, data)`
- Triggers de push:
 - Novo serviço na categoria/região do prestador → `NEW_SERVICE_NEARBY`
 - Lead desbloqueado → cliente recebe `LEAD_UNLOCKED`
 - Crédito adicionado → `CREDITS_ADDED`
 - Verificação aprovada/rejeitada → `VERIFICATION_RESULT`
- Service Worker `public/sw.js` com handler `push` e `notificationclick`
- **Aceitação:** prestador ativa push, cliente posta serviço, prestador recebe notificação.

TASK 14 — Painel Admin

- `/admin` dashboard com métricas:
 - Receita por dia / mês / ano (gráfico de barras)
 - Top 10 prestadores (por gasto)
 - Top categorias
 - Heatmap de bairros (Google Maps com pontos coloridos por densidade)
 - Funil: cadastros → onboarding completo → primeira compra
- `/admin/usuarios` — listar, banir, dar créditos manuais (`addManualCredits(userId, qty, reason)`)
- `/admin/transacoes` — listar todas Transactions com filtros
- `/admin/verificacoes` — fila de PENDING, botões aprovar/rejeitar
- `/admin/auditoria` — listar AuditLogs
- **Aceitação:** admin vê todas as métricas e consegue gerir users + verificações.

TASK 15 — Rate limiting (Upstash)

- Configurar Upstash Redis no `.env`
- `src/server/rate-limit.ts` com helper `checkRateLimit(key, limit, window)`
- Aplicar em:

- `createService` : 5/hora por user
- `unlockService` : 30/hora por user
- `buyCredits` : 10/hora por user
- `submitReport` : 5/dia por user
- Webhook InfinityPay: 100/min por IP
- **Aceitação:** ultrapassar limite retorna 429 com mensagem amigável.

TASK 16 — Email Resend

- Configurar Resend no `.env`
- Templates React Email:
 - Boas-vindas (com 5 créditos)
 - PIX gerado (com QR Code)
 - Pagamento confirmado
 - Lead desbloqueado (cliente)
 - Verificação aprovada/rejeitada
 - Estorno anti-vácuo
- **Aceitação:** todos os emails são enviados nos triggers corretos.

TASK 17 — TWA / Play Store

- Configurar `manifest.ts` completo (name, short_name, theme_color, icons, start_url)
- `public/.well-known/assetlinks.json` com SHA-256 do app TWA
- README com passo-a-passo Bubblewrap pra gerar APK
- **Aceitação:** PWA instalável no Android, Bubblewrap gera APK assinado.

TASK 18 — Docker + docker-compose + GitHub Actions

- `Dockerfile` multi-stage (deps → build → runner standalone Next.js)
- `docker-compose.yml` com:
 - service `web` (Next.js)

- `service redis` (opcional, se não usar Upstash gerenciado)
 - rede interna
 - volumes
 - healthcheck
 - `.github/workflows/deploy.yml` :
 - Build da imagem
 - Push para registry (ghcr.io)
 - SSH na VPS IntegraHost (secrets) → `docker compose pull && docker compose up -d`
 - **Aceitação:** push na main faz deploy automático na VPS.
-

10. RESTRIÇÕES NEGATIVAS — O QUE NÃO FAZER

- ❌ Não usar `any` em TypeScript
- ❌ Não criar API Routes quando Server Action resolve
- ❌ Não usar `useEffect` para fetching
- ❌ Não usar `Float` ou `Decimal` para dinheiro (sempre `Int` em centavos)
- ❌ Não criar seeders ou mocks de produção
- ❌ Não fazer cron jobs externos — toda lógica de expiração é lazy (em queries)
- ❌ Não armazenar telefone sem máscara E.164 (`+5564991234567`)
- ❌ Não permitir `creditBalance` negativo
- ❌ Não atualizar `creditBalance` fora de `prisma.$transaction` com Transaction registrada
- ❌ Não confiar em dados do client — todo input passa por Zod no server
- ❌ Não logar telefone, CPF ou tokens em `console.log` de produção
- ❌ Não expor o WhatsApp do cliente em nenhum lugar do mural (só após unlock)
- ❌ Não permitir admin alterar saldo sem motivo (campo `reason` obrigatório, vai pra `AuditLog`)

- ❌ Não duplicar créditos no webhook (idempotência via `pixTxId`)
 - ❌ Não permitir desbloqueio sem verificação aprovada
 - ❌ Não esquecer cabeçalhos de segurança (CSP, HSTS, X-Frame-Options) no `next.config.ts`
-

11. VARIÁVEIS DE AMBIENTE (.env.example)

```
# App
NEXT_PUBLIC_APP_URL=https://trampai.com.br
NODE_ENV=development

# Database (Supabase)
DATABASE_URL="postgresql://..."
DIRECT_URL="postgresql://..."

# NextAuth
AUTH_SECRET=
AUTH_URL=http://localhost:3000
AUTH_GOOGLE_ID=
AUTH_GOOGLE_SECRET=

# Supabase Storage
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# InfinityPay
INFINITYPAY_API_KEY=
INFINITYPAY_API_BASE_URL=
INFINITYPAY_WEBHOOK_SECRET=

# Google Maps
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=
GOOGLE_MAPS_SERVER_API_KEY=

# Resend
RESEND_API_KEY=
RESEND_FROM_EMAIL=Trampai <noreply@trampai.com.br>

# Upstash Redis
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=
```

```
# Web Push (gerar com `pnpm dlx web-push generate-vapid-keys`)
NEXT_PUBLIC_VAPID_PUBLIC_KEY=
VAPID_PRIVATE_KEY=
VAPID_SUBJECT=mailto:contato@trampai.com.br

# Admin (criar manualmente o primeiro)
INITIAL_ADMIN_EMAIL=
```

12. ESTRUTURA DE RESPOSTA EXIGIDA DO AGENTE

Para cada task, o agente deve responder neste formato:

```
## TASK XX — <nome>

### Arquivos criados
- src/...
- src/...

### Arquivos modificados
- src/...

### Comandos a executar
```bash
pnpm install ...
pnpm prisma migrate dev --name ...
```

### Testes manuais

1. Acessar X
2. Clicar em Y
3. Verificar Z

### Critério de aceitação

✅ <X> ✅ <Y>

### Pendências / Dúvidas

(qualquer coisa que precise de input do humano)

---

### ## 13. CHECKLIST FINAL – ANTES DE LANÇAR

- [ ] Todas as 18 tasks aceitas
- [ ] `pnpm build` sem erros nem warnings de tipo
- [ ] Lighthouse PWA  $\geq 90$
- [ ] Webhook InfinityPay testado com curl idempotente
- [ ] Backup automático Supabase configurado
- [ ] Domínio `trampai.com.br` apontado pra VPS
- [ ] Certificado SSL (Let's Encrypt via Traefik ou Caddy)
- [ ] Termos de uso e política de privacidade revisados
- [ ] Primeiro admin criado manualmente no banco
- [ ] Conta Google Play Developer com TWA submetido

---

**\*\*FIM DA ESPECIFICAÇÃO. Comece pela TASK 01.\*\***